

Digital Rehab App

Dokumentacja Techniczna Systemu

Autorzy

Bartek Czyż, Piotr Kotuła, Michał Patronik, Miłosz Śmieja

Wersja dokumentu:	1.0.0
Wersja aplikacji:	1.0.0+1
Data wydania:	czerwiec 2026
Język dokumentacji:	polski
Forma:	elektroniczna (PDF)

Projekt Zespołowy — PWr

Politechnika Wrocławska
Wydział Informatyki i Telekomunikacji

Typ dokumentu: Dokumentacja powykonawcza
Status: Wersja deweloperska

Dokumentacja sporządzona zgodnie z wymaganiami dot. dokumentacji systemów informatycznych (Załącz. nr 1.1 do SIWZ oraz Załącz. nr 6 do Umowy)

Spis treści

1	Informacje ogólne	3
1.1	Cel dokumentu	3
1.2	Zakres systemu	3
1.3	Wersjonowanie dokumentacji	3
2	Architektura systemu	3
2.1	Architektura ogólna	3
2.2	Schemat blokowy systemu	4
2.3	Komponenty frontendu (Flutter)	4
2.4	Komponenty backendu (Spring Boot)	5
3	Wymagania systemowe	5
3.1	Wymagania serwera (backend)	5
3.2	Wymagania urządzenia mobilnego (frontend)	5
4	Instalacja i konfiguracja	6
4.1	Baza danych MySQL	6
4.2	Konfiguracja backendu	6
4.3	Uruchomienie backendu	6
4.4	Konfiguracja i uruchomienie frontendu	7
5	Schemat bazy danych	7
5.1	Tabela users	7
5.2	Tabela exercises	8
5.3	Tabela exercise_history	8
5.4	Relacje między tabelami	8
6	Dokumentacja API	8
6.1	Informacje ogólne	8
6.2	Endpointy — Użytkownicy	9
6.2.1	Rejestracja nowego konta	9
6.2.2	Logowanie	9
6.2.3	Lista pacjentów trenera	9
6.2.4	Postępy pacjenta	9
6.3	Endpointy — Ćwiczenia	9
6.3.1	Lista ćwiczeń	9
6.3.2	Zapisanie wyniku treningu	10
6.3.3	Historia ćwiczeń użytkownika	10
7	Dokumentacja użytkownika	10
7.1	Uruchomienie i logowanie	10
7.2	Panel klienta	10
7.3	Panel trenera / fizjoterapeuty	11
8	Licencjonowanie	11
8.1	Oprogramowanie własne	11
8.2	Zależności zewnętrzne — Backend	11

8.3	Zależności zewnętrzne — Frontend	12
8.4	Zewnętrzne usługi / API	12
9	Procedury eksploatacyjne	12
9.1	Kolejność uruchamiania komponentów	12
9.2	Zatrzymanie systemu	12
9.3	Procedura backup i odtworzenia bazy danych	12
9.4	Weryfikacja działania systemu	13
9.5	Logi aplikacji	13
9.6	Resetowanie bazy danych do stanu początkowego	13
10	Testy jednostkowe backendu	13
10.1	Strategia testowania	13
10.2	Technologie i adnotacje	13
10.3	Uruchamianie testów	14
10.4	Pokryte scenariusze — UserControllerTest	14
10.5	Pokryte scenariusze — ExercisesControllerTest	15
10.6	Wyniki	15
11	Bezpieczeństwo	15
11.1	Zastosowane mechanizmy	15
11.2	Znane ograniczenia bezpieczeństwa	16
12	Polityka rozwoju oprogramowania	16
13	Spis dokumentów zewnętrznych	16
A	Pełna struktura katalogów projektu	17
B	Kody odpowiedzi HTTP	18

1 Informacje ogólne

1.1 Cel dokumentu

Niniejszy dokument stanowi kompletną dokumentację techniczną systemu **Digital Rehab App** — aplikacji mobilnej wspomagającej rehabilitację i ćwiczenia fizyczne. Dokumentacja obejmuje opis architektury systemu, instrukcje instalacji i konfiguracji, opis funkcjonalności, procedury eksploatacyjne oraz informacje o licencjonowaniu komponentów.

Dokument przeznaczony jest dla:

- administratorów systemu odpowiedzialnych za instalację i utrzymanie,
- użytkowników końcowych (klientów i trenerów/fizjoterapeutów),
- programistów rozwijających lub integrujących system z innymi rozwiązaniami.

1.2 Zakres systemu

Digital Rehab App jest systemem dwuwarstwowym składającym się z:

- **Aplikacji mobilnej** (frontend) — zbudowanej w technologii Flutter, uruchamianej na urządzeniach z systemem Android (API 24+),
- **Serwera REST API** (backend) — zbudowanego w technologii Spring Boot, komunikującego się z relacyjną bazą danych MySQL.

System umożliwia pacjentom samodzielne wykonywanie ćwiczeń rehabilitacyjnych z detekcją ruchu opartą na uczeniu maszynowym (Google ML Kit), a trenerom/fizjoterapeutom — monitorowanie postępów swoich podopiecznych.

1.3 Wersjonowanie dokumentacji

Nr wersji	Data	Autor	Opis zmian
1.0.0	2026-06	Zespół projektowy	Pierwsza wersja dokumentacji końcowej

2 Architektura systemu

2.1 Architektura ogólna

System Digital Rehab App oparty jest na architekturze **klient-serwer** z podziałem na trzy warstwy:

1. **Warstwa prezentacji** — aplikacja Flutter na urządzeniu mobilnym użytkownika,
2. **Warstwa logiki biznesowej** — serwer Spring Boot eksponujący REST API,

3. Warstwa danych — relacyjna baza danych MySQL.

Komunikacja między klientem a serwerem odbywa się poprzez protokół HTTP z wymianą danych w formacie JSON. Dane dotyczące historii treningów przechowywane są lokalnie na urządzeniu mobilnym w bazie Hive (No-SQL, offline-first) — synchronizacja z backendem jest planowana w kolejnej wersji systemu.

2.2 Schemat blokowy systemu

Urządzenie mobilne (Android) Flutter 3.41+ / Dart 3.11+
↓↑ HTTP REST (JSON) ↓↑
Serwer aplikacji Spring Boot 4.0.6 / Java 21 - localhost:8080
↓↑ JDBC / JPA (Hibernate) ↓↑
Baza danych MySQL 8 localhost:3306 / rehab_db

2.3 Komponenty frontendu (Flutter)

Komponent	Opis
lib/config/	Konfiguracja URL backendu i kluczy API
lib/core/	Motyw graficzny aplikacji
lib/data/models/	Modele danych (Exercise, Workout-Session)
lib/data/services/	Warstwa serwisów REST (ApiService)
lib/ui/screens/	Ekrany panelu klienta i trenera
lib/ui/widgets/	Wielokrotnie używane komponenty UI
lib/vision/	Silnik detekcji pozy (ML Kit + licznik powtórzeń)

2.4 Komponenty backendu (Spring Boot)

Komponent	Opis
controllers/	Warstwa REST: UsersController, ExercisesController
entities/	Encje JPA: Users, Exercises, Exercise-shHistory
repositories/ DataInitializer	Interfejsy Spring Data JPA Inicjalizacja bazy przykładowymi danymi
application.properties	Konfiguracja połączenia z bazą danych

3 Wymagania systemowe

3.1 Wymagania serwera (backend)

Komponent	Wersja minimalna	Rekomendowana
Java (JDK)	17	21 (LTS)
Maven	3.6	3.9+
MySQL	8.0	8.0
Docker	dowolna aktualna	dowolna aktualna
RAM	512 MB	1 GB
Dysk	500 MB	1 GB

3.2 Wymagania urządzenia mobilnego (frontend)

Komponent	Wymaganie	Uwagi
System operacyjny	Android API 24 (7.0+)	Wymagane do ML Kit
Flutter SDK	3.41+	Do kompilacji
Dart SDK	3.11+	Do kompilacji
Kamera	Wymagana	Detekcja pozy
RAM	min. 2 GB	3 GB rekomendowane

4 Instalacja i konfiguracja

4.1 Baza danych MySQL

Zalecanym sposobem uruchomienia bazy danych jest kontener Docker. Poniższe polecenie tworzy instancję MySQL 8:

Listing 1: Pierwsze uruchomienie bazy danych (Docker)

```
1 docker run -d \  
2   - --name rehab_mysql \  
3   -e MYSQL_ALLOW_EMPTY_PASSWORD=yes \  
4   -e MYSQL_DATABASE=rehab_db \  
5   -p 3306:3306 \  
6   mysql:8
```

Przy kolejnych uruchomieniach (kontener już istnieje):

Listing 2: Ponowne uruchomienie kontenera

```
1 docker start rehab_mysql
```

Alternatywnie, przy lokalnej instalacji MySQL bez Dockera:

Listing 3: Konfiguracja bez Dockera

```
1 sudo systemctl start mysql  
2 mysql -u root -e "CREATE DATABASE IF NOT EXISTS rehab_db;"
```

4.2 Konfiguracja backendu

Parametry połączenia z bazą danych znajdują się w pliku backend/src/main/resources

Listing 4: Domyślna konfiguracja application.properties

```
1 spring.application.name=aplikacja_wspomagajaca_cwiczenia  
2 spring.datasource.url=jdbc:mysql://localhost:3306/rehab_db  
3 spring.datasource.username=root  
4 spring.datasource.password=  
5 spring.jpa.hibernate.ddl-auto=update  
6 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.  
  MySQLDialect
```

Przy wdrożeniu na innym hoście należy zmienić wartość `spring.datasource.url` oraz uzupełnić dane uwierzytelniające.

4.3 Uruchomienie backendu

Listing 5: Uruchomienie serwera Spring Boot

```
1 cd backend  
2 chmod +x mvnw  
3 ./mvnw spring-boot:run
```

Serwer startuje na porcie 8080. Przy pierwszym uruchomieniu `DataInitializer` automatycznie tworzy schemat bazy (Hibernate DDL) i wypełnia ją danymi demonstracyjnymi.

4.4 Konfiguracja i uruchomienie frontendu

Przed kompilacją należy skopiować i uzupełnić plik konfiguracyjny:

Listing 6: Konfiguracja pliku `app_config.dart`

```
1 cp frontend/lib/config/app_config.example.dart \
2   frontend/lib/config/app_config.dart
```

Następnie w `app_config.dart` ustawić wartość `backendUrl`:

Scenariusz	Wartość <code>backendUrl</code>
Emulator Android	<code>http://10.0.2.2:8080</code>
Fizyczne urządzenie	<code>http://<IP_MASZYNY>:8080</code>

IP maszyny: `hostname -I | awk '{print $1}'`.

Dodatkowo należy uzupełnić `rapidApiKey` — klucz generowany na [RapidAPI \(ExerciseDB\)](#), wymagany do katalogu ćwiczeń.

Listing 7: Kompilacja i uruchomienie frontendu

```
1 cd frontend
2 flutter pub get
3 flutter run
4 # lub na konkretnym urządzeniu:
5 flutter devices
6 flutter run -d <AID_URZDZENIA>
```

5 Schemat bazy danych

5.1 Tabela `users`

Kolumna	Typ	Ograniczenia	Opis
<code>id</code>	BIGINT	PK, AUTO_INCREMENT	Identyfikator użytkownika
<code>email</code>	VARCHAR	UNIQUE, NOT NULL	Adres e-mail (login)
<code>password</code>	VARCHAR	NOT NULL	Hasło (patrz sekcja 11.2)
<code>role</code>	VARCHAR	NOT NULL	KLIENT lub TRENER
<code>trainer_id</code>	BIGINT	FK → <code>users(id)</code>	ID trenera (nullable)

5.2 Tabela exercises

Kolumna	Typ	Ograniczenia	Opis
id	BIGINT	PK, AUTO_INCREMENT	Identyfikator ćwiczenia
name	VARCHAR	NOT NULL	Nazwa ćwiczenia
description	VARCHAR	-	Opis i wskazówki wykonania

5.3 Tabela exercise_history

Kolumna	Typ	Ograniczenia	Opis
id	BIGINT	PK, AUTO_INCREMENT	Identyfikator wpisu
user_id	BIGINT	FK → users(id)	Kto ćwiczył
exercise_id	BIGINT	FK → exercises(id)	Jakie ćwiczenie
repetitions	INT	NOT NULL	Liczba powtórzeń
accuracy	INT	0-100	Poprawność (%)
created_at	DATETIME	DEFAULT NOW()	Data i czas sesji

5.4 Relacje między tabelami

- `users.trainer_id → users.id` — relacja wiele-do-jednego (klienci do trenera),
- `exercise_history.user_id → users.id` — historia przypisana do użytkownika,
- `exercise_history.exercise_id → exercises.id` — historia przypisana do ćwiczenia.

6 Dokumentacja API

6.1 Informacje ogólne

Parametr	Wartość
Protokół	HTTP
Format danych	JSON
Port	8080
Base URL	<code>http://<HOST>:8080</code>
Autentykacja	Brak (TODO: JWT)

6.2 Endpointy — Użytkownicy

6.2.1 Rejestracja nowego konta

Listing 8: POST /api/users/newAccount

```
1 POST /api/users/newAccount
2 Content-Type: application/json
3
4 {
5   "email": "jan@example.com",
6   "password": "haslo123",
7   "role": "KLIENT",
8   "trainer": { "id": 1 } // opcjonalne - ID trenera
9 }
```

Odpowiedzi: **200 OK** (sukces, zwraca obiekt użytkownika), **400 Bad Request** (niepoprawny email/hasło/rola lub nieistniejący trener), **409 Conflict** (email już zajęty).

6.2.2 Logowanie

Listing 9: POST /api/users/login

```
1 POST /api/users/login
2 Content-Type: application/json
3
4 { "email": "jan@example.com", "password": "haslo123" }
```

Odpowiedzi: **201 Created** (obiekt użytkownika z polem role), **400 Bad Request** (brak emaila), **403 Forbidden** (błędne hasło).

6.2.3 Lista pacjentów trenera

Listing 10: GET /trainers/{id}/clients

```
1 GET /trainers/{id}/clients
```

Zwraca listę użytkowników o roli KLIENT przypisanych do trenera.

6.2.4 Postępy pacjenta

Listing 11: GET /users/{id}/progress

```
1 GET /users/{id}/progress
```

6.3 Endpointy — Ćwiczenia

6.3.1 Lista ćwiczeń

Listing 12: GET /api/exercises

```
1 GET /api/exercises
```

6.3.2 Zapisanie wyniku treningu

Listing 13: POST /api/history

```

1 POST /api/history
2 Content-Type: application/json
3
4 {
5   "user":      { "id": 2 },
6   "exercise": { "id": 1 },
7   "repetitions": 15,
8   "accuracy":  87
9 }

```

6.3.3 Historia ćwiczeń użytkownika

Listing 14: GET /users/{id}/history

```

1 GET /users/{id}/history

```

7 Dokumentacja użytkownika

7.1 Uruchomienie i logowanie

Po uruchomieniu aplikacji wyświetlany jest ekran powitalny (WelcomeScreen), a następnie ekran logowania. Użytkownik podaje adres e-mail, hasło oraz wybiera rolę (Klient / Trener/Fizjoterapeuta). Po poprawnym zalogowaniu aplikacja przekierowuje do odpowiedniego panelu.

Dane demonstracyjne dostępne po instalacji:

Rola	E-mail	Hasło
Trener	trener@test.com	trener123
Klient	klient@test.com	klient123
Pacjent demo	marek.kowalski@demo.pl	demo1234
Pacjent demo	anna.nowak@demo.pl	demo1234
Pacjent demo	piotr.wisniewski@demo.pl	demo1234
Pacjent demo	katarzyna.wojcik@demo.pl	demo1234
Pacjent demo	tomasz.lewandowski@demo.pl	demo1234

7.2 Panel klienta

- **Katalog ćwiczeń** (CatalogScreen) — przeglądanie listy ćwiczeń z zewnętrznego API ExerciseDB oraz bazy lokalnej, filtrowanie po partii mięśniowej.
- **Szczegóły ćwiczenia** (ExerciseDetailScreen) — opis, wskazówki techniczne, przycisk rozpoczęcia sesji z kamerą.
- **Sesja treningowa z kamerą** (CameraScreen) — kamera analizuje ułożenie ciała w czasie rzeczywistym przy użyciu Google ML Kit Pose De-

tection. Silnik PushUpCounter zlicza powtórzenia na podstawie kąta stawu łokciowego (próg fazy dolnej: 110°, górnej: 160°).

- **Podsumowanie sesji** (SessionSummaryScreen) — liczba powtórzeń i wskaźnik poprawności po zakończeniu treningu.
- **Historia treningów** (WorkoutHistoryScreen) — lista minionych sesji z wykresami (fl_chart). Dane przechowywane lokalnie (Hive).

7.3 Panel trenera / fizjoterapeuty

- **Lista pacjentów** (PatientListScreen) — przegląd podopiecznych ze statusem: aktywny / nieobecny / zgłoszony ból.
- **Szczegóły pacjenta** (PatientDetailScreen) — telemetria, plan ćwiczeń, historia sesji pacjenta.
- **Edytor planu ćwiczeń** (PlanEditorScreen) — tworzenie i modyfikowanie indywidualnych planów rehabilitacyjnych.
- **Przegląd wideo** (VideoReviewScreen) — *w budowie* (planowane w kolejnej wersji).
- **Wiadomości** — *w budowie* (planowane w kolejnej wersji).

8 Licencjonowanie

8.1 Oprogramowanie własne

Kod źródłowy aplikacji (frontend Flutter i backend Spring Boot) jest własnością zespołu projektowego. Nie jest objęty żadną publiczną licencją open-source — wszelkie prawa zastrzeżone w zakresie projektu akademickiego.

8.2 Zależności zewnętrzne — Backend

Biblioteka	Wersja	Licencja
Spring Boot	4.0.6	Apache 2.0
Hibernate ORM	6.x	LGPL 2.1
MySQL Connector/J	8.x	GPL 2.0 (FOSS exception)
Lombok	1.18.x	MIT
Apache Commons Validator	1.10.1	Apache 2.0

8.3 Zależności zewnętrzne — Frontend

Pakiet	Wersja	Licencja
Flutter SDK	3.41+	BSD 3-Clause
google_mlkit_pose_detection	0.14.0	BSD / Google
camera	0.11.0	BSD 3-Clause
hive / hive_flutter	2.2.3	Apache 2.0
fl_chart	0.69.0	MIT
http	1.6.0	BSD 3-Clause
google_fonts	6.2.1	Apache 2.0
permission_handler	11.3.1	MIT

8.4 Zewnętrzne usługi / API

ExerciseDB (RapidAPI) — zewnętrzne API katalogu ćwiczeń. Wymaga klucza API (rapidApiKey). Licencja komercyjna z planem darmowym (limit zapytań miesięcznie).

9 Procedury eksploatacyjne

9.1 Kolejność uruchamiania komponentów

Kolejność jest istotna:

1. Uruchomić kontener MySQL: `docker start rehab_mysql`
2. Uruchomić backend: `./mvnw spring-boot:run` (katalog backend/)
3. Uruchomić aplikację mobilną: `flutter run` (katalog frontend/)

9.2 Zatrzymanie systemu

1. Zamknąć aplikację mobilną
2. Zatrzymać backend (Ctrl+C)
3. Zatrzymać MySQL: `docker stop rehab_mysql`

9.3 Procedura backup i odtworzenia bazy danych

Wykonanie kopii zapasowej:

Listing 15: Kopia zapasowa bazy danych

```
1 docker exec rehab_mysql mysqldump -u root rehab_db \
2   > backup_$(date +%Y%m%d).sql
```

Odtworzenie z kopii:

Listing 16: Odtworzenie bazy z kopii zapasowej

```
1 docker exec -i rehab_mysql mysql -u root rehab_db \
2   < backup_20260601.sql
```

Dane historii treningów przechowywane lokalnie na urządzeniu (Hive) można wyeksportować przez standardowy mechanizm kopii systemu Android (ADB backup).

9.4 Weryfikacja działania systemu

Listing 17: Weryfikacja działania API

```
1 # Pobranie listy cwiczen -- oczekiwana odpowiedz: tablica JSON
2 curl http://localhost:8080/api/exercises
```

9.5 Logi aplikacji

Listing 18: Uruchomienie backendu z zapisem logów

```
1 mkdir -p logs
2 ./mvnw spring-boot:run > logs/backend.log 2>&1 &
```

Logi Hibernate (DDL, SQL) dostępne na standardowym wyjściu konsoli serwera.

9.6 Resetowanie bazy danych do stanu początkowego

Listing 19: Reset bazy danych

```
1 docker stop rehab_mysql && docker rm rehab_mysql
2 docker run -d --name rehab_mysql \
3   -e MYSQL_ALLOW_EMPTY_PASSWORD=yes \
4   -e MYSQL_DATABASE=rehab_db \
5   -p 3306:3306 mysql:8
6 # DataInitializer załaduje dane po uruchomieniu backendu
7 ./mvnw spring-boot:run
```

10 Testy jednostkowe backendu

10.1 Strategia testowania

Warstwa testów jednostkowych backendu opiera się na izolowanym testowaniu kontrolerów REST przy użyciu adnotacji `@WebMvcTest`. Podejście to uruchamia wyłącznie warstwę sieciową (serwlet, routowanie, serializację JSON) bez inicjalizacji bazy danych — repozytoria zastępowane są atrapami za pomocą adnotacji `@MockitoBean`.

Testy pisane są według stylu BDD (*Behavior-Driven Development*): warunek wstępny konfigurowany przez `BDDMockito.given(...).willReturn(...)`, weryfikacja odpowiedzi przez `MockMvc` i asercje `jsonPath`.

10.2 Technologie i adnotacje

Technologia / adnotacja	Rola w testach
@WebMvcTest(XyzController.class)	Ładuje wyłącznie warstwę MVC wybranego kontrolera
@MockitoBean	Rejestruje atrapę repozytorium w kontekście Springa
MockMvc	Symuluje żądania HTTP bez uruchamiania serwera
BDDMockito.given()	Konfiguruje zachowanie atrapy (styl BDD)
jsonPath(\$.pole)	Weryfikuje pola w odpowiedzi JSON
JUnit 5 + @DisplayName	Czytelne nazwy testów po polsku

Uwaga Spring Boot 4.x: Adnotacja @MockBean (Spring Boot 3.x) została zastąpiona przez @MockitoBean (org.springframework.test.context.bean.override.mockito). Analogicznie @WebMvcTest pochodzi z nowego pakietu org.springframework.boot.web

10.3 Uruchamianie testów

Listing 20: Uruchomienie wszystkich testów jednostkowych

```
1 cd backend
2 ./mvnw test
```

Listing 21: Uruchomienie testów wybranego kontrolera

```
1 ./mvnw test -Dtest=UserControllerTest
2 ./mvnw test -Dtest=ExercisesControllerTest
```

Raporty XML i TXT zapisywane są do katalogu backend/target/surefire-reports/.

10.4 Pokryte scenariusze — UserControllerTest

Klasa testuje endpointy POST /api/users/newAccount, POST /api/users/login oraz GET /trainers/{id}/clients.

Scenariusz	Endpoint	Status
Rejestracja TRENERA z poprawnymi danymi	newAccount	200
Rejestracja KLIENTA z istniejącym trenerem	newAccount	200
Rejestracja KLIENTA bez trenera	newAccount	200
Niepoprawny format adresu e-mail	newAccount	400
Hasło krótsze niż 6 znaków	newAccount	400
Zduplikowany adres e-mail	newAccount	409
Nieprawidłowa rola (np. ADMIN)	newAccount	400
Klient podaje ID nieistniejącego trenera	newAccount	400
Klient podaje ID użytkownika bez roli TRENER	newAccount	400

Scenariusz	Endpoint	Status
Logowanie z poprawnymi danymi	login	201
Logowanie ze złym hasłem	login	403
Logowanie z nieznanym adresem e-mail	login	400
Logowanie bez pola email	login	400
Trener z przypisanymi klientami	clients	200
Trener bez klientów — pusta lista	clients	200

Odpowiedź logowania weryfikowana jest dodatkowo pod kątem obecności pól id, email, role oraz **braku** pola password (sprawdzenie działania @JsonProperty(W

10.5 Pokryte scenariusze — ExercisesControllerTest

Klasa testuje endpointy GET /api/exercises, POST /api/history oraz GET /users/{id}/history.

Scenariusz	Endpoint	Status
Pobranie listy ćwiczeń (dane istnieją)	GET /api/exercises	200
Pobranie ćwiczeń gdy baza pusta	GET /api/exercises	200
Dodanie wpisu historii z poprawnym body	POST /api/history	201
Pobranie historii ćwiczeń użytkownika	GET /users/{id}/history	200

10.6 Wyniki

Klasa testowa	Liczba testów	Wynik
UserControllerTest	15	PASS
ExercisesControllerTest	4	PASS
Łącznie	19	0 błędów, 0 pominięć

11 Bezpieczeństwo

11.1 Zastosowane mechanizmy

- **Walidacja danych wejściowych** — backend weryfikuje poprawność formatu adresu e-mail (Apache Commons Validator) oraz minimalną długość hasła (wyrażenie regularne `\w{6,}`).
- **Ochrona hasła w odpowiedziach API** — pole password w encji Users oznaczone adnotacją @JsonProperty(WRITE_ONLY), co uniemożliwia jego zwrot w odpowiedziach GET.

- **Walidacja ról i relacji** — backend sprawdza poprawność roli użytkownika i istnienie przypisywanego trenera przed zapisem.

11.2 Znane ograniczenia bezpieczeństwa

Poniższe braki zostały zidentyfikowane i są zaplanowane do eliminacji w kolejnych wersjach:

Lp.	Opis	Priorytet
1	Hasła przechowywane w bazie jako plaintext (brak BCrypt)	Wysoki
2	Brak mechanizmu sesji / tokenów JWT	Wysoki
3	Komunikacja HTTP zamiast HTTPS	Wysoki
4	Brak blokady konta po wielokrotnych błędnych logowaniach	Średni
5	Brak wygasania sesji	Średni
6	Historia treningów nie synchronizowana z backendem	Niski

Uwaga: System w obecnej postaci przeznaczony jest wyłącznie do użytku w środowisku deweloperskim i akademickim. Przed wdrożeniem produkcyjnym wymagane jest usunięcie wszystkich wyżej wymienionych ograniczeń.

12 Polityka rozwoju oprogramowania

System oparty jest na technologiach aktywnie wspieranych przez producentów:

- Flutter / Dart — Google; wsparcie LTS dla kanałów stable,
- Spring Boot 4.x — VMware/Broadcom; wsparcie do co najmniej 2027,
- MySQL 8.0 — Oracle; wsparcie do 2026 (Extended Support do 2030),
- Java 21 LTS — wsparcie do 2029 (OpenJDK / Eclipse Temurin).

Funkcje planowane w kolejnych wersjach: szyfrowanie haseł (BCrypt), autentykacja JWT, HTTPS, synchronizacja historii Hive ↔ backend, przegląd wideo sesji, moduł wiadomości trener-pacjent, obsługa kolejnych typów ćwiczeń w silniku detekcji pozy.

13 Spis dokumentów zewnętrznych

Dokument	Opis	Źródło
Spring Boot Docs	Dokumentacja frameworka Spring Boot 4.x	docs.spring.io
Flutter Docs	Dokumentacja SDK Flutter 3.x	docs.flutter.dev
ML Kit Pose Detection	Dokumentacja Google ML Kit	developers.google.com

Dokument	Opis	Źródło
ExerciseDB API	Dokumentacja zewnętrzne- go API ćwiczeń	rapidapi.com
Hive Docs	Dokumentacja bazy Hive dla Flutter	docs.hivedb.dev
MySQL 8.0 Docs	Dokumentacja MySQL Se- rver 8.0	dev.mysql.com
Docker Docs	Dokumentacja Docker Engi- ne	docs.docker.com

A Pełna struktura katalogów projektu

```

rehab_app/
├── frontend/
│   ├── lib/
│   │   ├── config/
│   │   ├── core/
│   │   ├── data/
│   │   │   ├── models/
│   │   │   └── services/
│   │   ├── ui/
│   │   │   ├── screens/
│   │   │   │   ├── coach/
│   │   │   │   │   ├── coach_main_screen.dart
│   │   │   │   │   ├── patient_list_screen.dart
│   │   │   │   │   ├── patient_detail_screen.dart
│   │   │   │   │   ├── plan_editor_screen.dart
│   │   │   │   │   └── video_review_screen.dart
│   │   │   │   ├── camera_screen.dart
│   │   │   │   ├── catalog_screen.dart
│   │   │   │   ├── exercise_detail_screen.dart
│   │   │   │   ├── login_screen.dart
│   │   │   │   ├── session_summary_screen.dart
│   │   │   │   └── workout_history_screen.dart
│   │   │   └── widgets/
│   │   └── vision/
│   └── pubspec.yaml
└── backend/
    ├── src/main/java/com/example/demo/
    │   ├── controllers/
    │   ├── entities/
    │   ├── repositories/
    │   └── DataInitializer.java

```

B Kody odpowiedzi HTTP

Kod	Znaczenie w kontekście API
200 OK	Operacja zakończona sukcesem
201 Created	Zasób utworzony (logowanie, zapis historii)
400 Bad Request	Nieprawidłowe dane wejściowe
403 Forbidden	Nieprawidłowe hasło
409 Conflict	Konflikt (e-mail już zajęty)